



MODULE 7

iOS Application Architecture

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 02 | iOS Development with Swift

Module 7 Roadmap

Six blocks. By the end: a refactored, testable, dependency-injected PNC Mobile feature.



MVC vs. MVVM

- The Massive View Controller Problem
- Extracting the View Model
- the “Zero UIKit Imports” Test



The Massive View Controller Problem

```
class AccountsViewController: UIViewController {  
    override func viewDidLoad() {  
        super.viewDidLoad()  
        URLSession.shared.dataTask(with: accountsURL) { data, _, error in  
            guard let data else { return }  
            let accounts = try? JSONDecoder().decode([Account].self, from: data)  
            DispatchQueue.main.async {  
                self.accounts = accounts ?? []  
                for account in self.accounts where account.balance < 0 {  
                    self.showOverdraftWarning(for: account)    // business rule  
                }  
                self.tableView.reloadData()                    // UI update  
            }  
        }.resume()  
    }  
    // ...plus DataSource, Delegate, formatting, error alerts, navigation...  
}
```



Model-View-ViewModel (MVVM)

- This is an architecture pattern that separates concerns into separate objects
- The **Model** implements the structure of the data being used in the screen
- The **View** displays the content for the screen
- The **ViewModel** implements all application logic for the screen
- There is no absolute consensus on what is MVVM you can find discussions/arguments over details about what should go where
- The overall goal is to separate data (Model), presentation (View), and logic (ViewModel)



MVVM: Extracting the View Model

```
import Foundation    // <- not UIKit

final class AccountsViewModel {
    private let repository: AccountsRepository
    private(set) var accounts: [Account] = []
    var onAccountsUpdated: (() -> Void)?

    init(repository: AccountsRepository) {
        self.repository = repository
    }
    func loadAccounts() async {
        accounts = (try? await repository.fetchAccounts()) ?? []
        onAccountsUpdated?()
    }
    var overdraftAccounts: [Account] {
        accounts.filter { $0.balance < 0 }
    }
}
```



Block 1 - MVC vs. MVVM

Skill Check ✓

- Name three distinct responsibilities crammed into the Massive View Controller example.
- What's the literal, checkable test for whether the view has leaked into the business logic layer?
- In the MVVM version, what's left for the view controller to actually do?



Clean Architecture & Layering

- The Dependency Rule
- Presentation, Domain & Data Layers
- Mapping Layers onto an iOS Project



The Dependency Rule

Presentation

Views, ViewModels - what the user sees and interacts with

depends on ↓

Domain

Business rules, use cases, models - the core of the app, no framework imports

depends on ↓

Data

Repositories, API clients, persistence - where data actually comes from

Dependencies point downward only. Domain never imports from Presentation or Data.



Mapping Layers onto a Real iOS Project



Presentation/

AccountListView.swift, AccountsViewModel.swift



Domain/

Account.swift, AccountsRepository.swift (protocol only)



Data/

RemoteAccountsRepository.swift, AccountsAPIClient.swift

Notice: the Domain layer defines AccountsRepository as a protocol. The Data layer provides the concrete implementation. This inversion is what makes the dependency point inward - Domain doesn't know Data exists.



Block 2 - Clean Architecture & Layering

Skill Check ✓

- State the Dependency Rule in one sentence.
- Which layer is allowed to import UIKit or SwiftUI, and which layer must never import either?
- Why does the Domain layer define AccountsRepository as a protocol rather than a concrete class?



The Coordinator Pattern

- Navigation Logic Trapped in View Controllers
- Extracting a Coordinator
- Who Owns the Flow



Coordinators in UIKit

- In UIKit, a coordinator has the following responsibilities:
- It controls a view's context
 - i.e. whether it is used inside of a UINavigationController, a UITabBarController or just as is
- It defines which transitions are performed when certain events occur
 - Such as presenting a new view controller or pushing it onto the current navigation stack
- It creates view controllers and manages the memory exchange between them
 - e.g. by creating new view models



The Problem: Navigation Logic Trapped in View Controllers

- This screen now has a hidden dependency on a screen that isn't its own - pure coupling introduced by where the navigation code happened to be written.
- AccountDetailViewController now has to know TransferViewController exists, how to construct it, and what happens when it finishes.

Code Bite

```
class AccountDetailViewController:
    UIViewController {
    @objc func transferButtonTapped() {
        let transferVC =
            TransferViewController(fromAccount: account)
        transferVC.onComplete = { [weak self] in
            self?.navigationController?
                .popToRootViewController(animated: true)
        }
        navigationController?
            .pushViewController(transferVC, animated:
true)
    }
}
```



The Coordinator: One Object Owns the Flow

- Define a function that will do the navigation in a delegate protocol

Code Bite

```
protocol AccountDetailDelegate: AnyObject {  
    func accountDetailDidRequestTransfer(  
        from account: Account)  
}
```



Coordinators in SwiftUI

- A coordinator in *SwiftUI* consists of **two components**:
 - a coordinator *view*
 - and a coordinator *object*
- In SwiftUI, we have two different responsibilities to be taken care of by a coordinator:
 - A view's context needs to be controlled by a View that is part of the view hierarchy
 - And we need to control a view's lifecycle
- This can be achieved easily with an ObservableObject
- We can easily pass it around by reference and make it conform to the Identifiable protocol.



Implementing The Coordinator

- AccountDetailViewController calls `delegate?.accountDetailDidRequestTransfer(from:)` and knows nothing else about what screen comes next.

Code Bite

```
final class AccountsCoordinator {  
    private let navController: UINavigationController  
  
    func showTransfer(from account: Account) {  
        let vc = TransferViewController(fromAccount:  
account)  
        vc.onComplete = { [weak self] in  
            self?.navController  
                .popToRootViewController(animated: true)  
        }  
        navController.pushViewController(vc, animated:  
true)  
    }  
}
```



Block 3 - The Coordinator Pattern

Skill Check ✓

- What hidden coupling does putting navigation code directly in a view controller create?
- After adopting a Coordinator, what does AccountDetailViewController know about TransferViewController?
- Who decides what screen comes next in a Coordinator-based flow?

